

gaussian-fit-injection-test

May 24, 2026

1 Signal Injection Test

Test discrimination power of spike search algorithm by injecting random Gaussian signals into HARPS observations and looking for them.

We will also look for better search algorithms to improve injected signal recovery.

```
[1]: %matplotlib widget
```

```
[2]: eso_login = "goodmanj"
from astroquery.eso import Eso
eso = Eso()
eso.login(eso_login)
```

INFO: Authenticating goodmanj on www.eso.org... [astroquery.eso.core]

INFO: Authentication successful! [astroquery.eso.core]

```
[3]: import sys
sys.path.append("..")
import optical_seti_functions
import seti_catalog_functions
import Gaussian_Injector
import csv
import random
import matplotlib.pyplot as plt
import numpy as np
```

```
[16]: import importlib
importlib.reload(Gaussian_Injector)
```

```
[16]: <module 'Gaussian_Injector' from 'c:\\Users\\goodm\\OneDrive\\Documents\\optical_seti\\injection\\Gaussian_Injector.py'>
```

```
[6]: # Pick 10 random targets from OSETI_targets.txt.
# Run this once to generate a list, then use the same list for all future runs.
with open("../OSETI_targets.txt") as f:
    dict_reader = csv.DictReader(f, delimiter='\t', fieldnames=["Target", "RA",
↪ "DEC", "SpType",
↪ "T_eff", "B", "V", "G", "Dist", "Explore", "products_ascii", "products_csv"])
```

```

    OSETI_targets_dict = list(dict_reader)
n_rows = len(OSETI_targets_dict)
target_rows = [random.randint(0,n_rows-1) for i in range(10)]
targets = [OSETI_targets_dict[i]["Target"] for i in target_rows]
targets

```

```

[6]: ['HD102300',
      'HD205471',
      'TYC7193-571-1',
      'HIP115332',
      'HIP31862',
      'GJ801',
      'GJ4077',
      'HD164249A',
      'WASP-43',
      'GJ3759']

```

```

[4]: # Use this particular list of targets every time this code is run
targets = ['HD72968',
          'HIP56489',
          'HD50060',
          'HD92588',
          'GJ3187',
          'HD95521',
          'HD218566',
          'GJ1065',
          'HIP1119',
          'HD207700']

```

```

[5]: # Download one observation of each star, as needed
star_files = [seti_catalog_functions.download_one_obs(target,eso) for target in
↳ targets]

```

```

using cached file
C:\Users\goodm\.astropy\cache\astroquery\Eso\ADP.2014-10-02T10_02_09.883.fits
using cached file
C:\Users\goodm\.astropy\cache\astroquery\Eso\ADP.2021-05-19T01_02_07.372.fits
using cached file
C:\Users\goodm\.astropy\cache\astroquery\Eso\ADP.2014-10-02T10_03_01.833.fits
using cached file
C:\Users\goodm\.astropy\cache\astroquery\Eso\ADP.2014-10-02T10_03_31.423.fits
using cached file
C:\Users\goodm\.astropy\cache\astroquery\Eso\ADP.2016-01-11T01_03_47.070.fits
using cached file
C:\Users\goodm\.astropy\cache\astroquery\Eso\ADP.2014-09-24T09_41_30.860.fits
using cached file
C:\Users\goodm\.astropy\cache\astroquery\Eso\ADP.2016-06-21T01_04_10.871.fits
using cached file

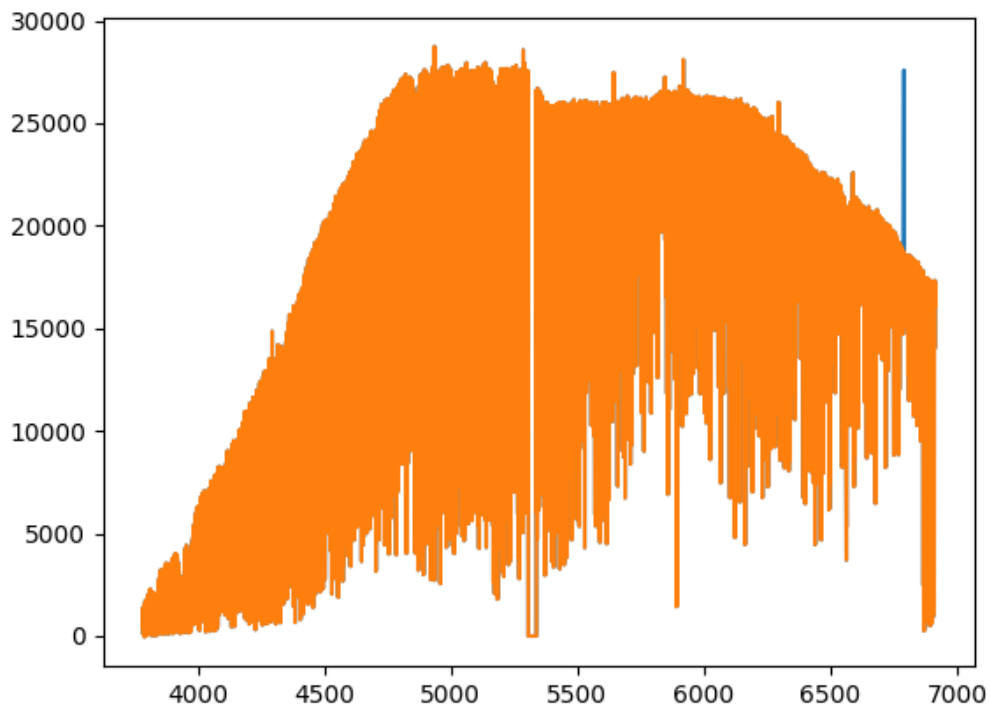
```

C:\Users\goodm\.astropy\cache\astroquery\Eso\ADP.2014-09-29T13_27_04.900.fits
 using cached file
 C:\Users\goodm\.astropy\cache\astroquery\Eso\ADP.2014-10-01T10_21_34.313.fits
 using cached file
 C:\Users\goodm\.astropy\cache\astroquery\Eso\ADP.2023-05-14T01_02_03.938.fits

```
[6]: # Plot one of the files with an example Gaussian injected.
[wave,arr1] = optical_seti_functions.read_harps_file(star_files[9][0])
array_length = len(arr1)
print(array_length)
center = random.randint(0,array_length)
print(center, wave[center])
modded_spectrum = Gaussian_Injector.
    ↪ add_gaussian_to_array(arr1,fwhm=10,center=float(center),array_length=array_length,area=1000)
plt.figure(1)
plt.clf()
plt.plot(wave,modded_spectrum, wave, arr1)
plt.show()
```

313044

300778 6790.51



Our injection code injects 10 spikes at random wavelengths into each of 10 star files. It counts the number of additional spikes it can detect. This test is done for various spike “area under the curve” values, and various full-width-half-maximum widths.

Statistics on the mean recovery rate, and its standard deviation among star files, are output.

```
[7]: # Main injection test. Inject n_injected gaussians into each of the
      ↪ observations, and count the percentage of spikes successfully recovered.
      # Do this for several areas[] under the Gaussian curve, and for several fwhms[].
      ↪ Store the average recovery rate for each area and fwhm in
      ↪ recovery_rate_means[],
      # and the standard deviation in recovery_rate_stdevs[].
n_injected = 3
areas = [10000,20000,100000]
fwhms = [4,10,20]
recovery_rate_means = np.zeros((len(areas),len(fwhms)))
recovery_rate_stdevs = np.zeros((len(areas),len(fwhms)))
for row in range(len(areas)):
    for col in range(len(fwhms)):
        area = areas[row]
        fwhm = fwhms[col]
        recovery_rates = []
        for s in star_files:
            print(s[0])
            [wave,arr1] = optical_seti_functions.read_harps_file(s[0])
            if len(arr1)==0:
                continue
            # Perform SETI spike analysis on original spectrum
            hits_start, hits_end,count = optical_seti_functions.
            ↪seti_spike_analyzer(arr1, min_count = 4, max_count = 60,
            ↪threshold_multiplier = 3.5, window_size = 101)
            count = len(hits_start)
            print(s[1],count)
            # Inject 10 random Gaussian signals
            injected_recovery_count = 0
            for n in range(n_injected):
                center = random.randint(0,array_length)
                modded_spectrum = Gaussian_Injector.add_gaussian_to_array(arr1,
            ↪fwhm = float(10), amplitude=None, center=float(center),
            ↪array_length=array_length, axis=-1, area=float(area))
                hits_start, hits_end, newcount = optical_seti_functions.
            ↪seti_spike_analyzer(modded_spectrum, min_count = 4, max_count = 60,
            ↪threshold_multiplier = 3.5, window_size = 1001)
                injected_recovery_count = injected_recovery_count
            ↪+len(hits_start) - count

            recovery_rates.append(float(injected_recovery_count)/n_injected)
```

```

        recovery_rate_means[row,col] = np.mean(recovery_rates)
        recovery_rate_stdevs[row,col] = np.std(recovery_rates)
print(recovery_rate_means)
print(recovery_rate_stdevs)

```

```

C:\Users\goodm\astropy\cache\astroquery\Eso\ADP.2014-10-02T10_02_09.883.fits
HR-3398 0
C:\Users\goodm\astropy\cache\astroquery\Eso\ADP.2021-05-19T01_02_07.372.fits
BD-123458 0

```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[7], line 28
    26     center = random.randint(0,array_length)
    27     modded_spectrum = Gaussian_Injector.add_gaussian_to_array(arr1, fwhm
↳ float(10), amplitude=None, center=float(center), array_length=array_length,
↳ axis=-1, area=float(area))
--> 28     hits_start, hits_end, newcount =
↳ optical_seti_functions.seti_spike_analyzer(modded_spectrum, min_count = 4, max_count = 60,
    29     injected_recovery_count = injected_recovery_count +len(hits_start)
↳ count
    31 recovery_rates.append(float(injected_recovery_count)/n_injected)

File c:\Users\goodm\OneDrive\Documents\optical_seti\injection\..
↳ \optical_seti_functions.py:142, in seti_spike_analyzer(arr1, min_count,
↳ max_count, threshold_multiplier, window_size, percentile)
    140 def seti_spike_analyzer(arr1, min_count = 4, max_count = 8,
↳ threshold_multiplier = 3.5, window_size = 101,percentile=0):
    141     half_window_size = round((window_size-1)/2)
--> 142     continuum = running_median(arr1, window_size)
    143     count = 0 #reset bright pixel count variable
    144 #     import numpy as np # JCG: Not needed

File c:\Users\goodm\OneDrive\Documents\optical_seti\injection\..
↳ \optical_seti_functions.py:41, in running_median(data, window_size)
    40 def running_median(data, window_size):
--> 41     return
↳ np.median(np.lib.stride_tricks.sliding_window_view(data>window_size),1)

File ~\AppData\Roaming\Python\Python311\site-packages\numpy\lib\function_base.p
↳ 3927, in median(a, axis, out, overwrite_input, keepdims)
    3845 @array_function_dispatch(_median_dispatcher)
    3846 def median(a, axis=None, out=None, overwrite_input=False,
↳ keepdims=False):
    3847     """
    3848     Compute the median along the specified axis.
    3849     (...)

```

```

3925
3926     """
-> 3927     return
↳ _ureduce(a, func=_median, keepdims=keepdims, axis=axis, out=out,
3928         overwrite_input=overwrite_input)

File ~\AppData\Roaming\Python\Python311\site-packages\numpy\lib\function_base.py :
↳ 3823, in _ureduce(a, func, keepdims, **kwargs)
3820         index_out = (0, ) * nd
3821         kwargs['out'] = out[(Ellipsis, ) + index_out]
-> 3823 r = func(a, **kwargs)
3825 if out is not None:
3826     return out

File ~\AppData\Roaming\Python\Python311\site-packages\numpy\lib\function_base.py :
↳ 3960, in _median(a, axis, out, overwrite_input)
3958     part = a
3959 else:
-> 3960     part = partition(a, kth, axis=axis)
3962 if part.shape == ():
3963     # make 0-D arrays work
3964     return part.item()

File ~\AppData\Roaming\Python\Python311\site-packages\numpy\core\fromnumeric.py
↳ 771, in partition(a, kth, axis, kind, order)
769 else:
770     a = asanyarray(a).copy(order="K")
--> 771 a.partition(kth, axis=axis, kind=kind, order=order)
772 return a

KeyboardInterrupt:

```

Results:

Mean recovery rate:

	FWHM=4	10	20
Area=10000	.19	.14	.16
20000	.24	.25	.2
100000	.47	.50	.47

Stdev of recovery rate:

	FWHM=4	10	20
Area=10000	.31	.26	.26
20000	.28	.32	.31

	FWHM=4	10	20
100000	.27	.36	.30

Recovery is mediocre and highly spotty. Some files give >70% recovery rate while others give zero. Density and depth of spectral lines in the star spectrum seems to be a major factor.

I don't see a clear sign that FWHM matters much: even with FWHM=20, recovery is poor.

One issue with our method is that the spike increases the local standard deviation, so our threshold rises near a spike. We can reduce that effect by using a wider averaging window. Let's try with 10x bigger window:

```
[ ]: # Repeat injection test with a wider moving average window for calculating
      ↪ medians and standard deviations (window_size=1001 instead of 101)
n_injected = 10
areas = [10000,20000,100000]
fwhms = [4,10,20]
recovery_rate_means = np.zeros((len(areas),len(fwhms)))
recovery_rate_stdevs = np.zeros((len(areas),len(fwhms)))
for row in range(len(areas)):
    for col in range(len(fwhms)):
        area = areas[row]
        fwhm = fwhms[col]
        recovery_rates = []
        for s in star_files:
            print(s[0])
            [wave,arr1] = optical_seti_functions.read_harps_file(s[0])
            if len(arr1)==0:
                continue
            # Perform SETI spike analysis on original spectrum
            hits_start, hits_end,count = optical_seti_functions.
            ↪seti_spike_analyzer(arr1, min_count = 4, max_count = 60,
            ↪threshold_multiplier = 3.5, window_size = 1001)
            count = len(hits_start)
            print(s[1],count)
            # Inject 10 random Gaussian signals
            injected_recovery_count = 0
            for n in range(n_injected):
                center = random.randint(0,array_length)
                modded_spectrum = Gaussian_Injector.add_gaussian_to_array(arr1,
            ↪fwhm = float(10), amplitude=None, center=float(center),
            ↪array_length=array_length, axis=-1, area=float(area))
                hits_start, hits_end, newcount = optical_seti_functions.
            ↪seti_spike_analyzer(modded_spectrum, min_count = 4, max_count = 60,
            ↪threshold_multiplier = 3.5, window_size = 1001)
                injected_recovery_count = injected_recovery_count
            ↪+len(hits_start) - count
```

```

        recovery_rates.append(float(injected_recovery_count)/n_injected)
    recovery_rate_means[row,col] = np.mean(recovery_rates)
    recovery_rate_stdevs[row,col] = np.std(recovery_rates)
print(recovery_rate_means)
print(recovery_rate_stdevs)

```

2 Wider Window Test

Results: the wider running median/stdev window N=1001 significantly improves injection recapture, at the expense of 10x greater analysis time.

Mean:

	FWHM=4	10	20
Area=10000	.26	.34	.25
20000	.38	.42	.4
100000	.77	.79	.81

Stdev:

	FWHM=4	10	20
Area=10000	.32	.34	.32
20000	.35	.43	.39
100000	.28	.27	.31

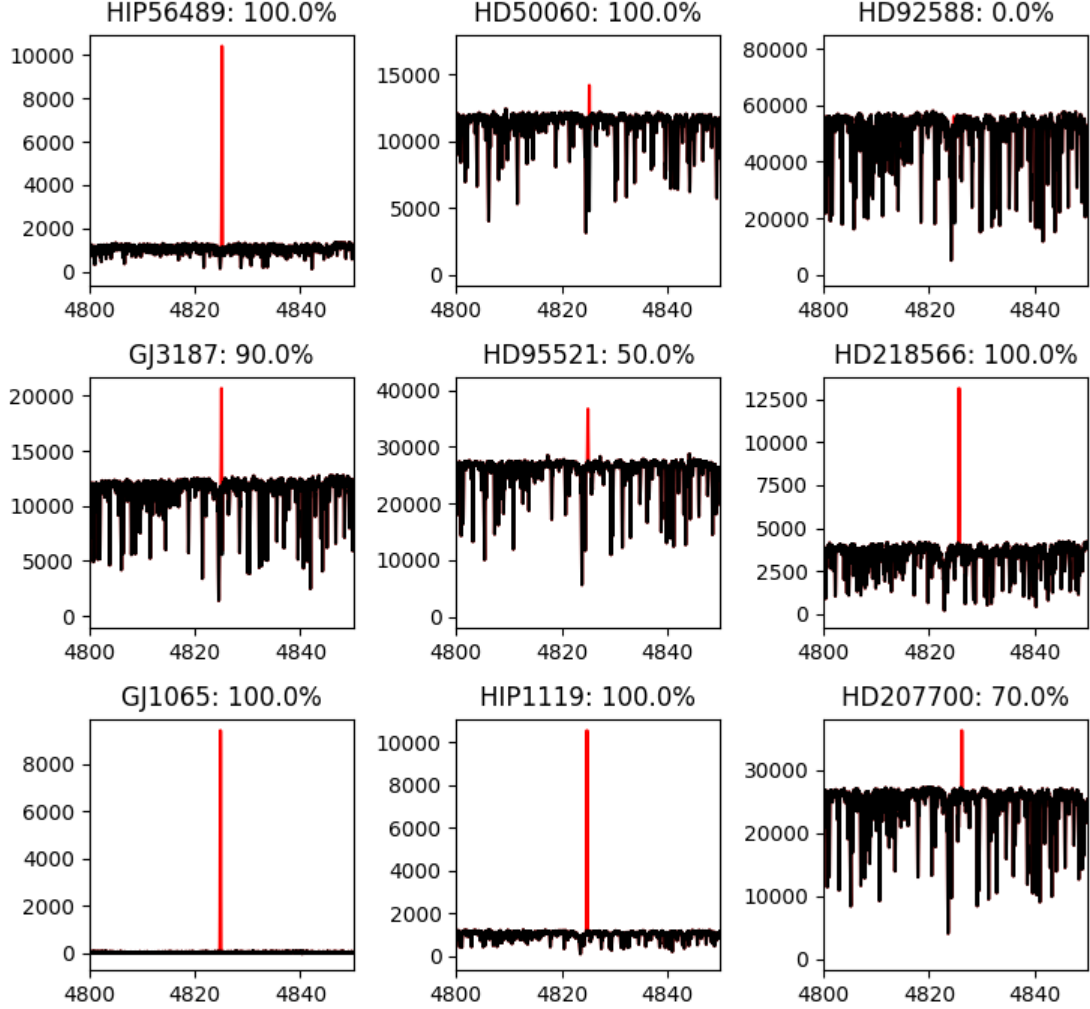
Calculation time = 45 minutes

Next, we ask: what are the properties of the stellar spectra where injected spikes cannot be easily recovered? Below, we plot several spectra along with their spike recovery rates. An example spike (FWHM=10, area=100,000) has been inserted.

```

[ ]: plt.figure(3)
plt.clf()
recovery_rates
for i in range(1,len(recovery_rates)):
    plt.subplot(3,3,i)
    [wave,arr1] = optical_seti_functions.read_harps_file(star_files[i][0])
    modded_spectrum = Gaussian_Injector.
    ↪add_gaussian_to_array(arr1,fwhm=10,center=array_length/
    ↪3,array_length=array_length,area=100000)
    plt.plot(wave,modded_spectrum,'r',wave,arr1,'k')
    plt.xlim([4800,4850])
    plt.title(targets[i]+' : '+str(recovery_rates[i]*100)+'%')
plt.tight_layout()
plt.show()

```



We are failing to recover injected spectra from *bright* stars, because our injected spikes (which have a fixed flux) are small compared to the threshold set by the high standard deviation of the bright star's strong absorption lines. Thus, we should look for an alternative threshold which is less sensitive to strong absorption features.

3 Percentile-based threshold

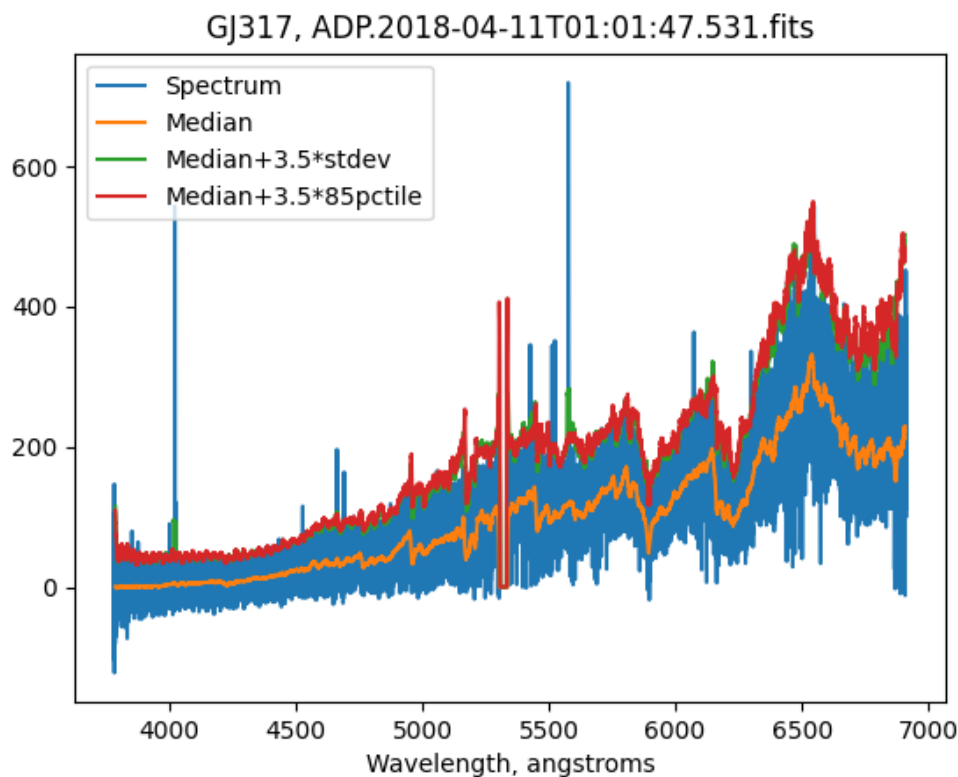
Instead of using the running standard deviation, let's try using the distance between the running median and the running 85th percentile. For Gaussian-distributed noise, this amounts to the same thing, since 85% of Gaussian random values lie beneath $\bar{x} + \sigma_x$. But for spectral data with a non-Gaussian distribution, the 85th percentile will run along the spectral continuum, and will be less sensitive to the extreme low values found in absorption features.

Thus, new our threshold is:

$$x_t = \text{running_median}(x) + 3.5 * [\text{running_percentile}(x, 85) - \text{running_median}(x)]$$

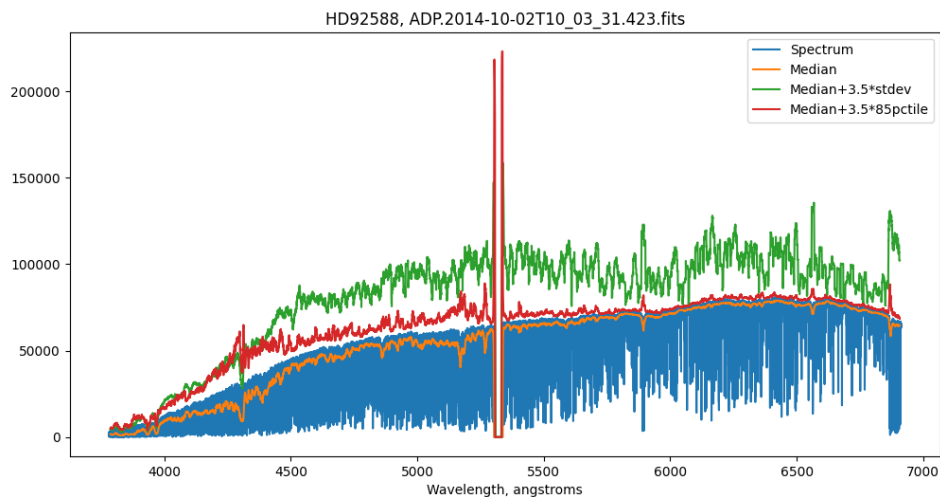
Let's compare it against the standard deviation method.

```
[ ]: file = 'ADP.2018-04-11T01:01:47.531.fits'
file = file.replace(":", "_")
[wave,arr1] = optical_seti_functions.read_harps_file(file)
running_median = np.median(np.lib.stride_tricks.
    ↳sliding_window_view(arr1,1001),1)
running_std = np.std(np.lib.stride_tricks.sliding_window_view(arr1,1001),1)
running_85p = np.percentile(np.lib.stride_tricks.
    ↳sliding_window_view(arr1,1001),85,1)
threshold_std = running_median + 3.5*running_std
threshold_85p = running_median + 3.5*(running_85p-running_median)
wave_trunc = wave[500:-500]
plt.figure(4)
plt.clf()
plt.
    ↳plot(wave,arr1,wave_trunc,running_median,wave_trunc,threshold_std,wave_trunc,threshold_85p)
plt.xlabel('Wavelength, angstroms')
plt.title('GJ317, ADP.2018-04-11T01:01:47.531.fits')
plt.legend(['Spectrum', 'Median', 'Median+3.5*stdev', 'Median+3.5*85pctile'])
plt.show()
```



This file has noise, but no strong absorption features, and we see that the percentile-based threshold is almost identical to the stdev-based one. Either would easily detect the SETI spike in this file at 4663 angstroms.

```
[ ]: file = 'ADP.2014-10-02T10_03_31.423.fits'
file = file.replace(":", "_")
[wave,arr1] = optical_seti_functions.read_harps_file(file)
running_median = np.median(np.lib.stride_tricks.
    ↳sliding_window_view(arr1,1001),1)
running_std = np.std(np.lib.stride_tricks.sliding_window_view(arr1,1001),1)
running_85p = np.percentile(np.lib.stride_tricks.
    ↳sliding_window_view(arr1,1001),85,1)
threshold_std = running_median + 3.5*running_std
threshold_85p = running_median + 3.5*(running_85p-running_median)
wave_trunc = wave[500:-500]
plt.figure(5)
plt.clf()
plt.
    ↳plot(wave,arr1,wave_trunc,running_median,wave_trunc,threshold_std,wave_trunc,threshold_85p)
plt.xlabel('Wavelength, angstroms')
plt.title('HD92588, ADP.2014-10-02T10_03_31.423.fits')
plt.legend(['Spectrum', 'Median', 'Median+3.5*stdev', 'Median+3.5*85pctile'])
plt.show()
```



This spectrum has strong absorption features, so the stdev-based threshold (green) is very high, but the percentile-based threshold (red) is much lower, increasing sensitivity.

Let's see how well our injection recovery code works with this new threshold. I've edited `optical_seti_functions.seti_spike_analyzer()` to optionally use a percentile-based threshold.

```
[ ]: importlib.reload(optical_seti_functions)
```

```
[ ]: <module 'optical_seti_functions' from 'c:\\Users\\goodm\\OneDrive\\Documents\\optical_seti\\injection\\..\\optical_seti_functions.py'>
```

```
[ ]: # Test on GJ317
```

```
file = 'ADP.2018-04-11T01:01:47.531.fits'
file = file.replace(":", "_")
[wave,arr1] = optical_seti_functions.read_harps_file(file)
# Original threshold used in thesis
hits_start, hits_end, count = optical_seti_functions.seti_spike_analyzer(arr1,
    ↪min_count = 4, max_count = 60, threshold_multiplier = 3.5, window_size = 101)
print(wave[hits_start])
# Std-based threshold with wider window size
hits_start, hits_end, count = optical_seti_functions.seti_spike_analyzer(arr1,
    ↪min_count = 4, max_count = 60, threshold_multiplier = 3.5, window_size =
    ↪1001)
print(wave[hits_start])
# percentile-based threshold with wider window size
hits_start, hits_end, count = optical_seti_functions.seti_spike_analyzer(arr1,
    ↪min_count = 4, max_count = 60, threshold_multiplier = 3.5, window_size =
    ↪1001, percentile=85)
print(wave[hits_start])
```

```
[4662.77]
```

```
[4527.05 4662.76 5512.62 6851.19]
```

```
[4527.05 4662.76 5512.62 6666.83]
```

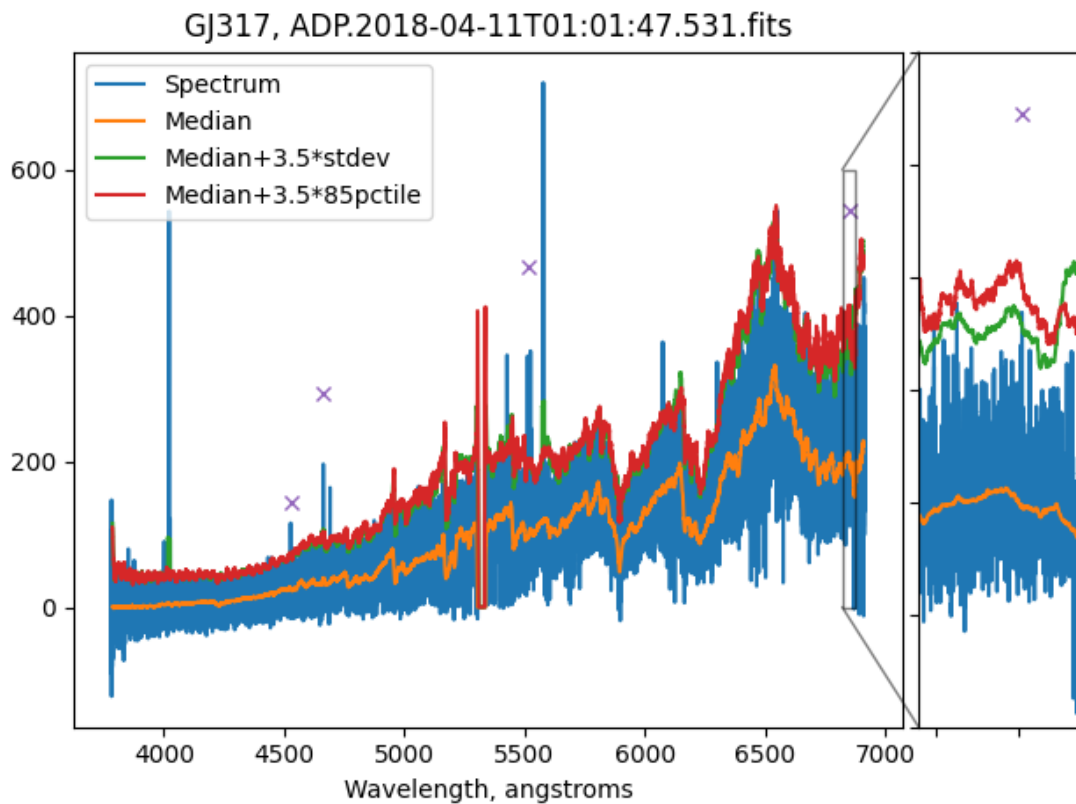
The percentile threshold always recovers the same spike at 4663 angstroms we already found, but it can return more, especially if the window width is wide. Are these “real” new spikes?

```
[ ]: file = 'ADP.2018-04-11T01:01:47.531.fits'
file = file.replace(":", "_")
[wave,arr1] = optical_seti_functions.read_harps_file(file)
running_median = np.median(np.lib.stride_tricks.
    ↪sliding_window_view(arr1,1001),1)
running_std = np.std(np.lib.stride_tricks.sliding_window_view(arr1,1001),1)
running_85p = np.percentile(np.lib.stride_tricks.
    ↪sliding_window_view(arr1,1001),85,1)
threshold_std = running_median + 3.5*running_std
threshold_85p = running_median + 3.5*(running_85p-running_median)
wave_trunc = wave[500:-500]
spike_indices = np.array([74543, 88114, 173100, 306957]) # Spikes identified in
    ↪previous step
plt.figure(6)
plt.clf()
```

```

plt.
    ↪plot(wave,arr1,wave_trunc,running_median,wave_trunc,threshold_std,wave_trunc,threshold_85p,
    ↪wave[spike_indices],1.5*arr1[spike_indices+2],'x')
plt.xlabel('Wavelength, angstroms')
plt.title('GJ317, ADP.2018-04-11T01:01:47.531.fits')
plt.legend(['Spectrum','Median','Median+3.5*stdev','Median+3.5*85pctile'])
ax = plt.gca()
axins = ax.inset_axes([1.02, 0.0, 0.2, 1.
    ↪0],xlim=(6820,6870),ylim=(0,600),xticklabels=[],yticklabels=[])
axins.
    ↪plot(wave,arr1,wave_trunc,running_median,wave_trunc,threshold_std,wave_trunc,threshold_85p,
    ↪wave[spike_indices],1.5*arr1[spike_indices+2],'x')
ax.indicate_inset_zoom(axins, edgecolor="black")
plt.tight_layout()
plt.show()

```



The Xes in the figure above mark the new spikes identified by the percentile threshold. They generally do correspond to interesting potential SETI candidates, though a zoom-in on the feature at 6700 angstroms shows it's probably just noise. A criterion of 4 or 5 times the 85th percentile might give better results.

```
[ ]: file = 'ADP.2018-04-11T01:01:47.531.fits'
file = file.replace(":", "_")
[wave,arr1] = optical_seti_functions.read_harps_file(file)
# percentile-based threshold with wider window size and higher threshold
↳ multiplier
hits_start, hits_end,count = optical_seti_functions.seti_spike_analyzer(arr1,
↳ min_count = 4, max_count = 60, threshold_multiplier = 5, window_size =
↳ 1001,percentile=85)
print(wave[hits_start])
# percentile-based threshold with narrow window size
hits_start, hits_end,count = optical_seti_functions.seti_spike_analyzer(arr1,
↳ min_count = 4, max_count = 60, threshold_multiplier = 3.5, window_size =
↳ 101,percentile=85)
print(wave[hits_start])
```

[4662.77 5512.63]

[4662.76 5512.62]

4 Injection Test with Percentile Threshold

Now let's see how well the injection test works using the $3.5 \times (85\text{th percentile})$ threshold. Repeating the earlier analysis:

```
[ ]: # Main injection test. Inject n_injected gaussians into each of the
↳ observations, and count the percentage of spikes successfully recovered.
# Do this for several areas[] under the Gaussian curve, and for several fwhms[].
↳ Store the average recovery rate for each area and fwhm in
↳ recovery_rate_means[],
# and the standard deviation in recovery_rate_stdevs[].
n_injected = 10
areas = [10000,20000,100000]
fwhms = [4,10,20]
recovery_rate_means = np.zeros((len(areas),len(fwhms)))
recovery_rate_stdevs = np.zeros((len(areas),len(fwhms)))
for row in range(len(areas)):
    for col in range(len(fwhms)):
        area = areas[row]
        fwhm = fwhms[col]
        recovery_rates = [];
        for s in star_files:
            print(s[0])
            [wave,arr1] = optical_seti_functions.read_harps_file(s[0])
            if len(arr1)==0:
                continue
            # Perform SETI spike analysis on original spectrum
```

```

        hits_start, hits_end, count = optical_seti_functions.
↪seti_spike_analyzer(arr1, min_count = 4, max_count = 60,
↪threshold_multiplier = 3.5, window_size = 1001, percentile=85)
        count = len(hits_start)
        print(s[1], count)
        # Inject 10 random Gaussian signals
        injected_recovery_count = 0
        for n in range(n_injected):
            center = random.randint(0, array_length)
            modded_spectrum = Gaussian_Injector.add_gaussian_to_array(arr1,
↪fwhm = float(10), amplitude=None, center=float(center),
↪array_length=array_length, axis=-1, area=float(area))
            hits_start, hits_end, newcount = optical_seti_functions.
↪seti_spike_analyzer(modded_spectrum, min_count = 4, max_count = 60,
↪threshold_multiplier = 3.5, window_size = 1001, percentile=85)
            injected_recovery_count = injected_recovery_count
↪+len(hits_start) - count

        recovery_rates.append(float(injected_recovery_count)/n_injected)
        recovery_rate_means[row,col] = np.mean(recovery_rates)
        recovery_rate_stdevs[row,col] = np.std(recovery_rates)
print(recovery_rate_means)
print(recovery_rate_stdevs)

```

Results:

The 3.5*85percentile threshold can reliably detect large spikes 90-95% of the time, regardless of their width.

Mean:

	FWHM=4		
	10	20	
Area=10000	.44	.48	.46
20000	.65	.71	.71
100000	.94	.94	.91

Stdev:

	FWHM=4		
	10	20	
Area=10000	.37	.33	.36
20000	.30	.30	.30
100000	.10	.12	.11

Calculation time = 65 minutes (!!!)

5 Number of Spikes Detected

How does the new threshold change the number of spikes that must be analyzed by hand? Returning to our random sample of 10 stars:

```
[11]: # Search Test. Count the of natural spikes in our 10 stars. Do this for both
      ↪the original threshold (3.5 times the standard deviation with a running
      ↪window of 101)
      # and new (3.5 times the 85th percentile range, with a running window of 1001)

for s in star_files:
    [wave,arr1] = optical_seti_functions.read_harps_file(s[0])
    # Perform SETI spike analysis on original spectrum
    old_hits_start, old_hits_end,count = optical_seti_functions.
    ↪seti_spike_analyzer(arr1, min_count = 4, max_count = 60,
    ↪threshold_multiplier = 3.5, window_size = 101)
    new_hits_start, new_hits_end,count = optical_seti_functions.
    ↪seti_spike_analyzer(arr1, min_count = 4, max_count = 60,
    ↪threshold_multiplier = 3.5, window_size = 1001,percentile=85)
    new75_hits_start, new_hits_end,count = optical_seti_functions.
    ↪seti_spike_analyzer(arr1, min_count = 4, max_count = 60,
    ↪threshold_multiplier = 7.5, window_size = 1001,percentile=85)
    print("%s old: %d, new: %d, new75:
    ↪%d"%(s[1],len(old_hits_start),len(new_hits_start),len(new75_hits_start)))

# And finally for GJ317, a star with known spikes:
file = 'ADP.2018-04-11T01:01:47.531.fits'
file = file.replace(":", "_")
[wave,arr1] = optical_seti_functions.read_harps_file(file)
# Perform SETI spike analysis on original spectrum
old_hits_start, old_hits_end,count = optical_seti_functions.
    ↪seti_spike_analyzer(arr1, min_count = 4, max_count = 60,
    ↪threshold_multiplier = 3.5, window_size = 101)
new_hits_start, new_hits_end,count = optical_seti_functions.
    ↪seti_spike_analyzer(arr1, min_count = 4, max_count = 60,
    ↪threshold_multiplier = 3.5, window_size = 1001,percentile=85)
new75_hits_start, new_hits_end,count = optical_seti_functions.
    ↪seti_spike_analyzer(arr1, min_count = 4, max_count = 60,
    ↪threshold_multiplier = 7.5, window_size = 1001,percentile=85)
print("%s old: %d, new: %d, new75:
    ↪%d"("GJ317",len(old_hits_start),len(new_hits_start),len(new75_hits_start)))
```

```
HR-3398 old: 0, new: 0, new7: 0
BD-123458 old: 0, new: 3, new7: 0
HD50060 old: 0, new: 6, new7: 0
HD92588 old: 0, new: 0, new7: 0
HD17970 old: 0, new: 1, new7: 0
HD95521 old: 0, new: 5, new7: 0
```

```

HD218566 old: 0, new: 1, new7: 0
GJ1065 old: 0, new: 7, new7: 3
BD-110028 old: 0, new: 4, new7: 0
HD 207700 old: 0, new: 3, new7: 0
GJ317 old: 1, new: 4, new75: 1

```

Yes, we're getting a *lot* more hits. This is a problem, but not an unexpected one: the new algorithm is better able to detect our injected spikes, so it's picking up more natural spikes too. A better automated airglow rejection system may help.

```
[18]: importlib.reload(optical_seti_functions)
```

```
[18]: <module 'optical_seti_functions' from 'c:\\Users\\goodm\\OneDrive\\Documents\\optical_seti\\injection\\..\\optical_seti_functions.py'>
```

6 Gaussian curve fitting: Injection recovery test

Let's inject Gaussian signals, retrieve them, and then do a Gaussian curve fit to compare their FWHM to the FWHM of the injected signal.

```
[28]: from astropy.table import QTable
```

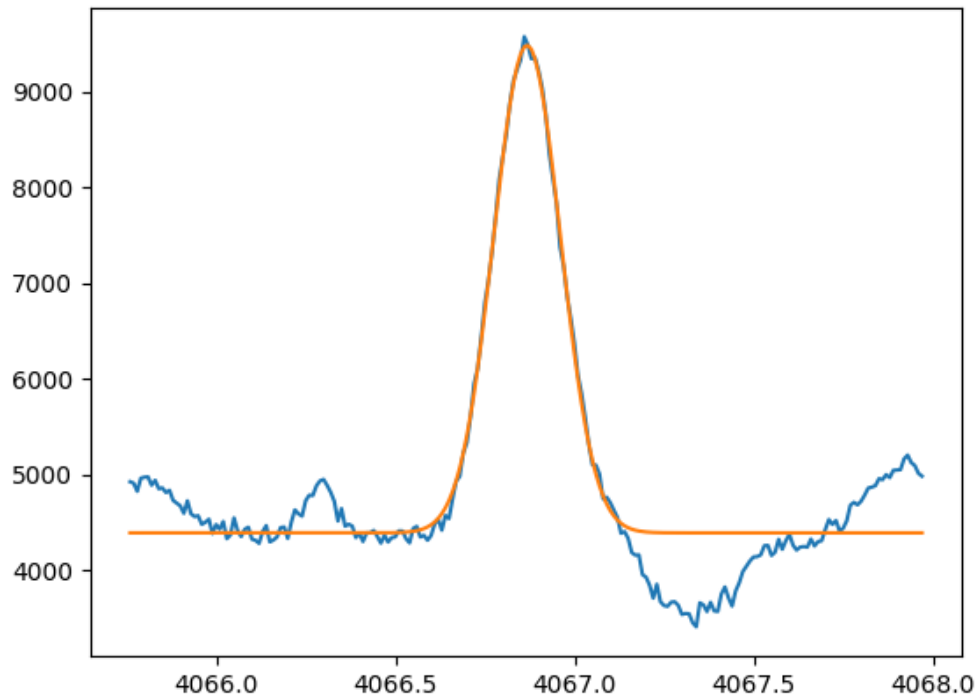
First let's test it with just one injected Gaussian:

```
[112]: [wave,arr1] = optical_seti_functions.read_harps_file(star_files[0][0])
hits_start_noinject, hits_end_noinject, count_noinject = optical_seti_functions.
    ↳ seti_spike_analyzer(arr1, min_count = 4, max_count = 60,
    ↳ threshold_multiplier = 7.5, window_size = 1001, percentile=85)
center = random.randint(0, array_length)
modded_spectrum = Gaussian_Injector.add_gaussian_to_array(arr1, fwhm =
    ↳ fwhm_injected, amplitude=None, center=float(center),
    ↳ array_length=array_length, axis=-1, area=area_injected)
hits_start_inject, hits_end_inject, count_inject = optical_seti_functions.
    ↳ seti_spike_analyzer(modded_spectrum, min_count = 4, max_count = 60,
    ↳ threshold_multiplier = 7.5, window_size = 1001, percentile=85)
hits_start = [x for x in hits_start_inject if x not in hits_start_noinject]
hits_end = [x for x in hits_end_inject if x not in hits_end_noinject]
plt.figure(2)
fwhm_recovered = optical_seti_functions.
    ↳ gaussian_curve_fit(wave, modded_spectrum, hits_start[0], hits_end[0], plot=True)
print(fwhm_recovered)
```

```
0.21965970150088393
```

WARNING: AstropyDeprecationWarning: The Spectrum1D class is deprecated and may be removed in a future version.

Use Spectrum instead. [optical_seti_functions]



0.21965970150088393

Now for all ten stars, three amplitudes, and three fwhms:

```
[114]: # Injection test with Gaussian fitting. Inject n_injected gaussians into each
      ↪ of the observations, and count the percentage of spikes successfully
      ↪ recovered.
      # Do this for several areas[] under the Gaussian curve, and for several fwhms[].
      ↪ Fit a Gaussian to each detected signal,
      # and record the detected fwhm.

injection_tests = {'file': [], 'target': [], 'area': [], 'input_fwhm': [], 'detected':
      ↪ [], 'output_fwhm': []}
n_injected = 10
areas = [10000, 20000, 100000]
fwhms = [4, 10, 20]
for row in range(len(areas)):
    for col in range(len(fwhms)):
        print(areas[row], fwhms[col])
        area_injected = float(areas[row])
        fwhm_injected = float(fwhms[col])
```

```

for s in star_files:
    [wave,arr1] = optical_seti_functions.read_harps_file(s[0])
    if len(arr1)==0:
        continue
    # Perform SETI spike analysis on original spectrum
    hits_start_noinject, hits_end_noinject, count_noinject =
    ↳optical_seti_functions.seti_spike_analyzer(arr1, min_count = 4, max_count =
    ↳60, threshold_multiplier = 3.5, window_size = 1001, percentile=85)
    # Inject 10 random Gaussian signals
    for n in range(n_injected):
        center = random.randint(0, array_length)
        modded_spectrum = Gaussian_Injector.add_gaussian_to_array(arr1,
    ↳fwhm = fwhm_injected, amplitude=None, center=float(center),
    ↳array_length=array_length, axis=-1, area=area_injected)
        hits_start_inject, hits_end_inject, count_inject =
    ↳optical_seti_functions.seti_spike_analyzer(modded_spectrum, min_count = 4,
    ↳max_count = 60, threshold_multiplier = 3.5, window_size = 1001, percentile=85)
        # Keep track of only the new hits that *don't* occur in the
    ↳control run.
        hits_start = [x for x in hits_start_inject if x not in
    ↳hits_start_noinject]
        hits_end = [x for x in hits_end_inject if x not in
    ↳hits_end_noinject]
        # Add to output dictionary
        injection_tests['file'].append(s[0])
        injection_tests['target'].append(s[1])
        injection_tests['area'].append(areas[row])
        injection_tests['input_fwhm'].append(wave[center+int(fwhms[col]/
    ↳2)]-wave[center-int(fwhms[col]/2)])
        if len(hits_start) >= 1:
            if len(hits_start) > 1:
                print("Found more signals than I injected!")
                injection_tests['detected'].append(True)
                fwhm_recovered = optical_seti_functions.
    ↳gaussian_curve_fit(wave, modded_spectrum, hits_start[0], hits_end[0], plot=False)
                injection_tests['output_fwhm'].append(fwhm_recovered)
            else:
                injection_tests['detected'].append(False)
                injection_tests['output_fwhm'].append(0)
# Store results in an astropy table
t =
    ↳QTable([injection_tests['area'], injection_tests['input_fwhm'], injection_tests['output_fwhm'],

```

10000 4

WARNING: AstropyDeprecationWarning: The Spectrum1D class is deprecated and may be removed in a future version.

Use Spectrum instead. [optical_seti_functions]

```
10000 10
```

```
WARNING: The fit may be unsuccessful; check:
```

```
The maximum number of function evaluations is exceeded.
```

```
[astropy.modeling.fitting]
```

```
10000 20
```

```
20000 4
```

```
20000 10
```

```
20000 20
```

```
Found more signals than I injected!
```

```
Found more signals than I injected!
```

```
Found more signals than I injected!
```

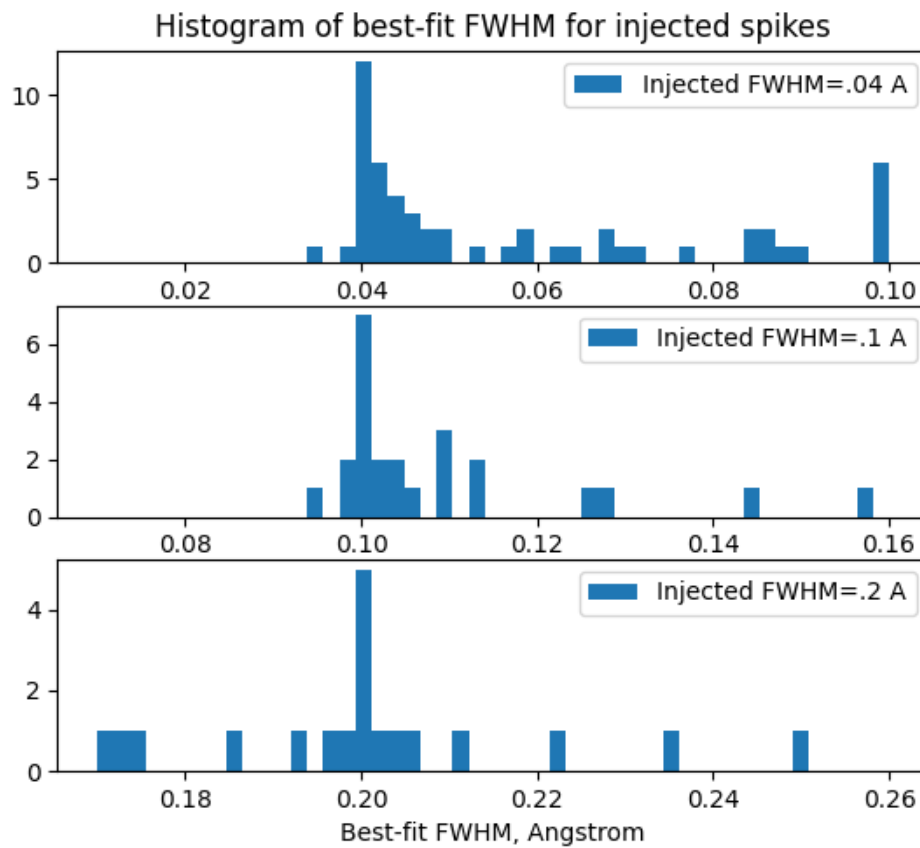
```
100000 4
```

```
100000 10
```

```
100000 20
```

```
[103]: mask = (t['input_fwhm'] < .05)
t_04 = t[mask]
mask = (t['input_fwhm'] > .05) & (t['input_fwhm'] < .12)
t_10 = t[mask]
mask = (t['input_fwhm'] > .12)
t_20 = t[mask]
```

```
[128]: plt.figure(5)
plt.clf()
bins = np.linspace(.01,.1,50)
plt.subplot(3,1,1)
plt.hist(t[(t['area'] == 10000)]['output_fwhm'],bins=bins)
plt.title('Histogram of best-fit FWHM for injected spikes')
plt.legend(['Injected FWHM=.04 A'])
plt.subplot(3,1,2)
plt.hist(t_10['output_fwhm'],bins=bins+.06)
plt.legend(['Injected FWHM=.1 A'])
plt.subplot(3,1,3)
plt.hist(t_20['output_fwhm'],bins=bins+.16)
plt.legend(['Injected FWHM=.2 A'])
plt.xlabel('Best-fit FWHM, Angstrom')
plt.show()
```



```
[130]: plt.savefig("gaussian-fit-injection.pdf",format="pdf")
```

```
[ ]:
```